

MolmoAct2 · arXiv 2605.02881 · Ai2 / allenai

Visionary

11+3 · all seats empty

Three outer loops for 2026

Keep FM · choose S2 · open substrate · RTC wrap?

Oct 28 paper club · VLA Architectures · quiet build

Spine: geometric S2 $\neq$ semantic HL $\neq$ stage voting · FM motor

FALSE: “has $\pi_{0.5}$ HL” / “has RTC” / “has BEHAVIOR S2 voting”

Open Action Reasoning Model (ARM): geometric System-2

(adaptive depth tokens in Think) before a per-layer-KV DiT FM action expert

— **deployable open alternative to closed π -series.**

Backbone

Molmo2-ER
4B + SigLIP2

Curriculum

FAST AR →
FM expert (KI) → FT

Expert

~621M DiT
L=36 · N=10 Euler

Think

100 depth codes
adaptive reuse

Claims tattoo: LIBERO 97.2 / Think 98.1 ($\pi_{0.5}$ 96.9) · Spaces 37.7 vs 34.5 · DROID OOD 87.1%

Rates: ~55.8 Hz Act2 / ~12.7 Hz Think @ H100 H=10 + CUDA Graphs · RTC absent

CRITICAL SPINE — three outer loops (do NOT conflate)

All three call an FM chunk expert (System 1 / motor). Different intermediate discrete states.

MolmoAct2 Think

Adaptive depth / geometry

- 10×10 depth codes (100)
- VQ depth lattice
- Regen if RGB cos < 0.996
- Cached static cells
- Geometric System-2

NOT semantic subtasks

$\pi_{0.5}$ HL

Semantic subtask language

- AR subtask strings
- Optional boxes
- Task language hierarchy
- Then FM LL chunk
- Semantic System-2

NOT depth lattice

BEHAVIOR'25 S2

Stage IDs + voting

- Task-progress stages
- Classifier + voting
- Non-Markov memory
- Then FM expert
- Checklist System-2

NOT scene geometry

FALSE claims: “has $\pi_{0.5}$ HL” · “has RTC” · “has BEHAVIOR S2 voting”

TENTATIVE: BEHAVIOR may want depth + stage complementary · RTC absent — latency via FM cache + CUDA Graphs + Think reuse

Visionary bets — 2026 BEHAVIOR / architecture

1 Keep FM low-level

Discrete AR actions slower (14 Hz vs 56 Hz).
Aligns FAST $\neq$ FM-at-deploy.

2 Choose S2 deliberately

Semantic HL/stages vs depth ARM vs hybrid.
Geometry $\neq$ task progress.

3 Open substrate when π gated

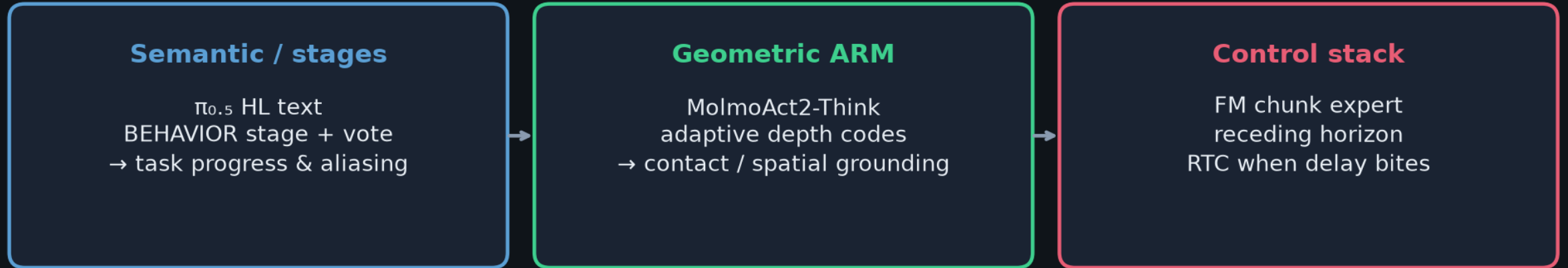
Don't assume OOTB wins BEHAVIOR without
stage tracking + RTC.

4 RTC wrap on Molmo FM

High-leverage follow-on · TENTATIVE.
Async soft-inpaint sibling to Think reuse.

Hybrid System-2 bet — geometry + stages

TENTATIVE best-of-breed for long-horizon household



2026 synthesis

Keep challenge-proven hierarchy/control (stages + RH + RTC).
Trial Molmo-style geometric tokens only where latency allows.
Never confuse amortized chunk Hz with closed-loop competence.

L0

Motion / Perception + language **control hierarchy — L0 → L1 → L2**

SigLIP2 + LLM multi-cam

Interface: tokens + per-layer K/V → expert



L1

System-2 (optional Think)

AR depth codes (10×10)

Interface: depth → VLM context → expert KV



L2

Motor

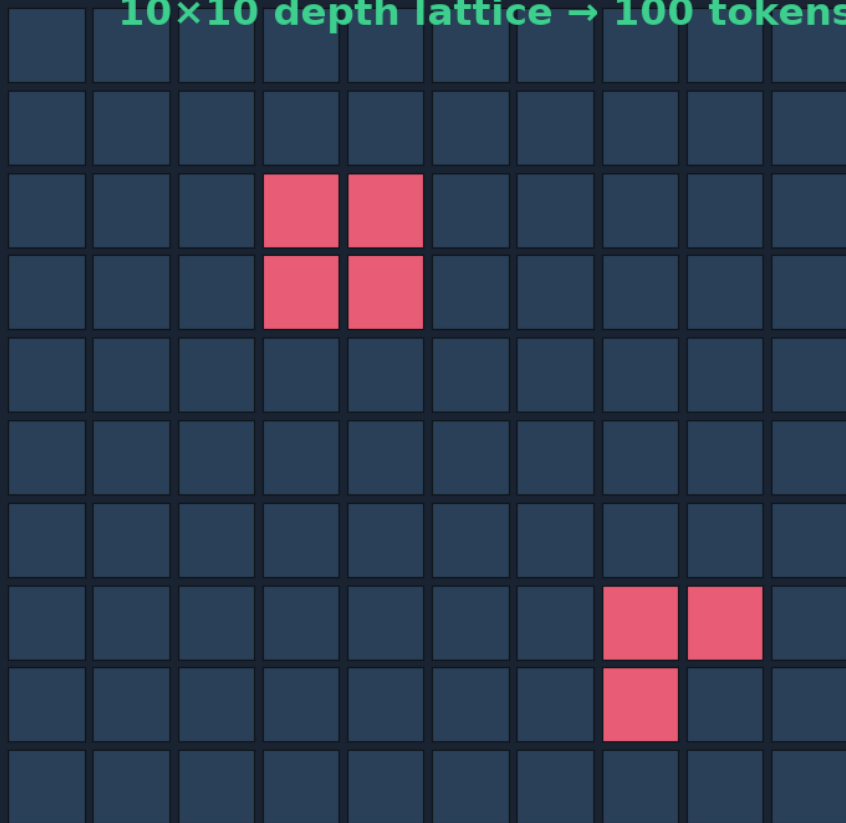
DiT FM · $H \approx H_z$ · N=10 Euler

Interface: continuous joints / delta-EE

MolmoAct2-Think — adaptive depth codes (geometric S2)

NOT $\pi_{0.5}$ semantic HL · NOT BEHAVIOR stage IDs

10×10 depth lattice → 100 tokens



pink = regenerate (RGB cos < 0.996) · blue-gray = cache reuse

1. Observe RGB (+ multi-cam)

2. AR depth codes <depth_0...127>

3. Reuse static cells / regen changed

4. Optional per-layer depth gate

5. Gate into expert KV → FM chunk

Latency tax: 12.7 Hz Think vs 55.8 Hz flat (H100, H=10, Graphs)

Control checklist — Empiricist / Visionary bridge

1. Quote SR only with (policy, think/HL/stage, H, OL|RH, NFE, Hz, ms, GPU)
2. Do not treat Think Hz as “solved real-time” without Graph/cache flags
3. Prefer Comet-style receding when porting to BEHAVIOR (paper is open-loop)
4. If testing “System 2,” pick ONE ontology per ablation cell
5. Don’t cargo-cult open-loop on long-horizon household tasks
6. RTC absent in MolmoAct2 — treat wrap as follow-on experiment (TENTATIVE)

Fig.4 · Architecture: VLM + per-layer KV → FM expert

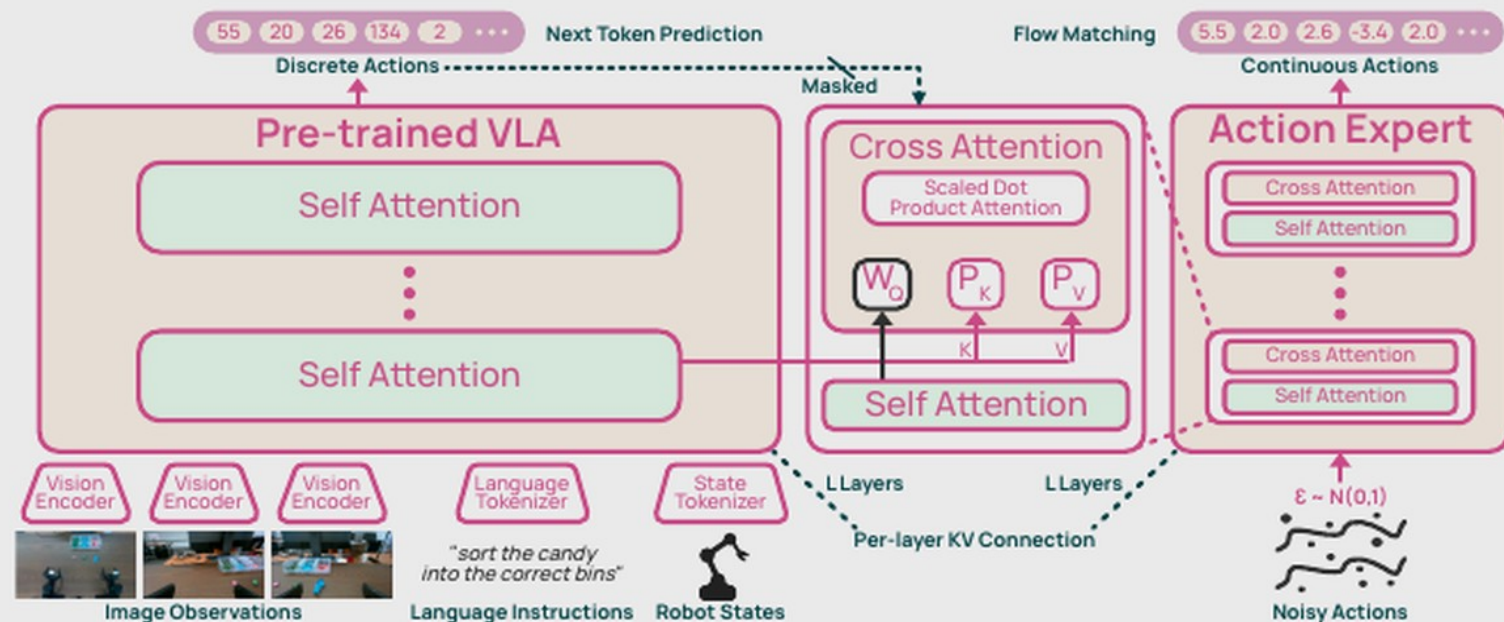


Figure 4 Overview of MolmoAct2. Image observations, language instructions, and robot states are tokenized and processed by a pre-trained VLA backbone with self-attention layers. Post-training attaches a DiT-style action expert with the same number of layers as the backbone. At each layer, the backbone key and value tensors are projected and reused as the key and value inputs to the action expert’s cross-attention, enabling per-layer KV conditioning from visual-language context to continuous control. The expert is trained with flow matching to denoise a noisy action trajectory into a continuous robot trajectory. During training, the backbone is also supervised with next-token prediction over discrete action tokens, while the target action-token span is masked from the expert so continuous action prediction cannot condition on the ground-truth discrete actions.

control strings using the same special-token wrappers used during training, e.g., `<setup_start>bimanual yam robotic arms in molmoact2<setup_end>`, and `<control_start>absolute joint pose<control_end>` or `<control_start>delta end-effector pose<control_end>`; the corresponding chat template and output trigger formatting are detailed in Appendix B.

Action and state representation. This stage is deliberately limited to discrete autoregressive supervision: the model predicts action tokens, but does not yet train the continuous action expert used in post-training. Each

Fig.5 · MolmoAct2-Think adaptive depth

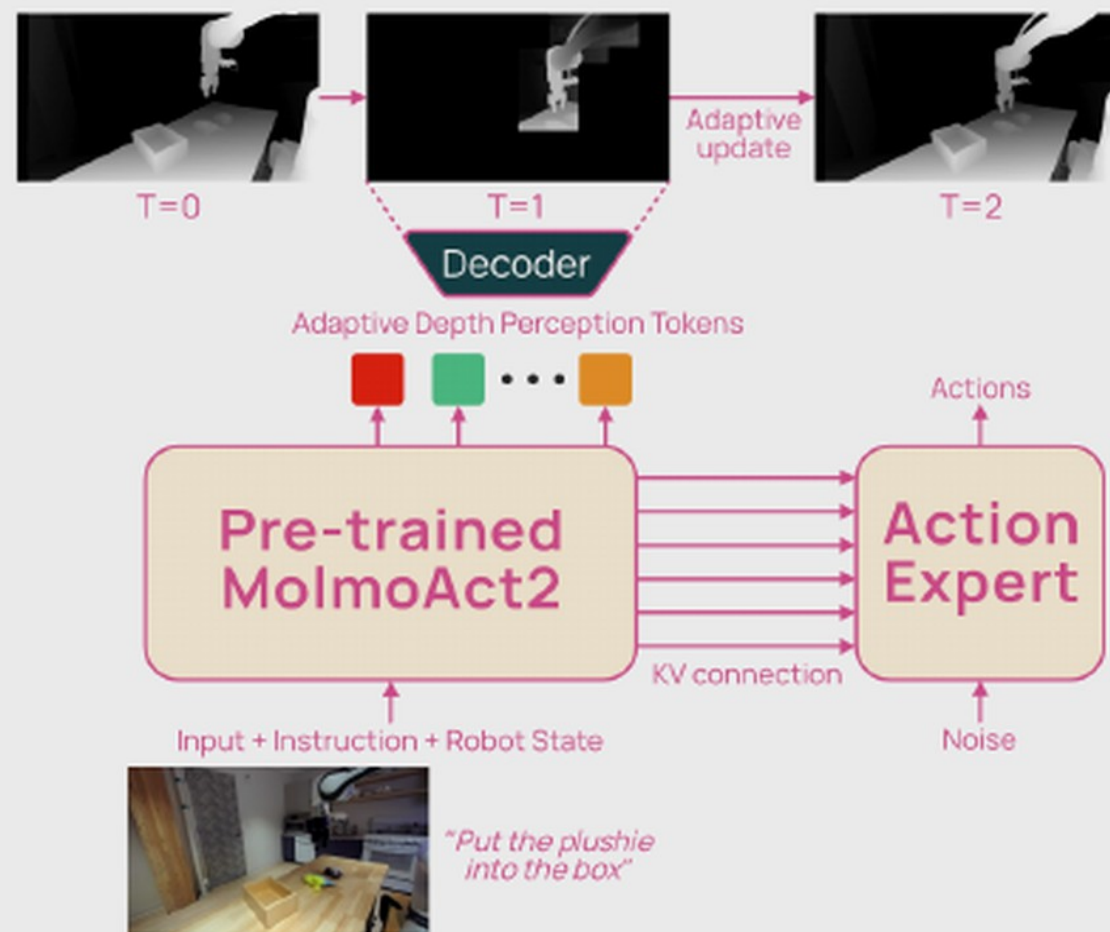
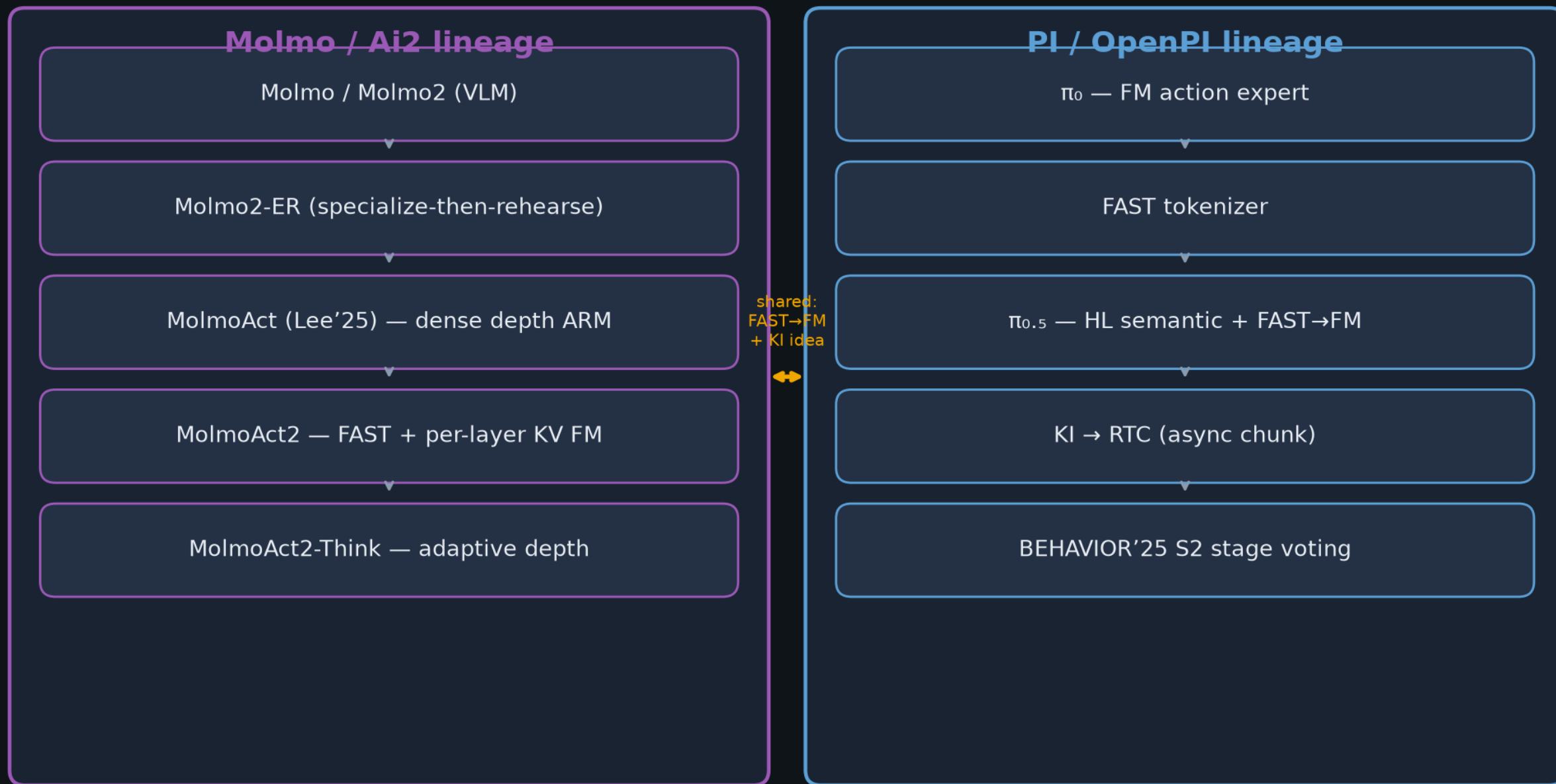


Figure 5 Overview of MolmoAct2-Think. MOLMOACT2-THINK augments the MOLMOACT2 action-generation pipeline with adaptive depth-token reasoning, reusing cached depth codes for static regions and regenerating depth codes for changed regions before conditioning the action expert.

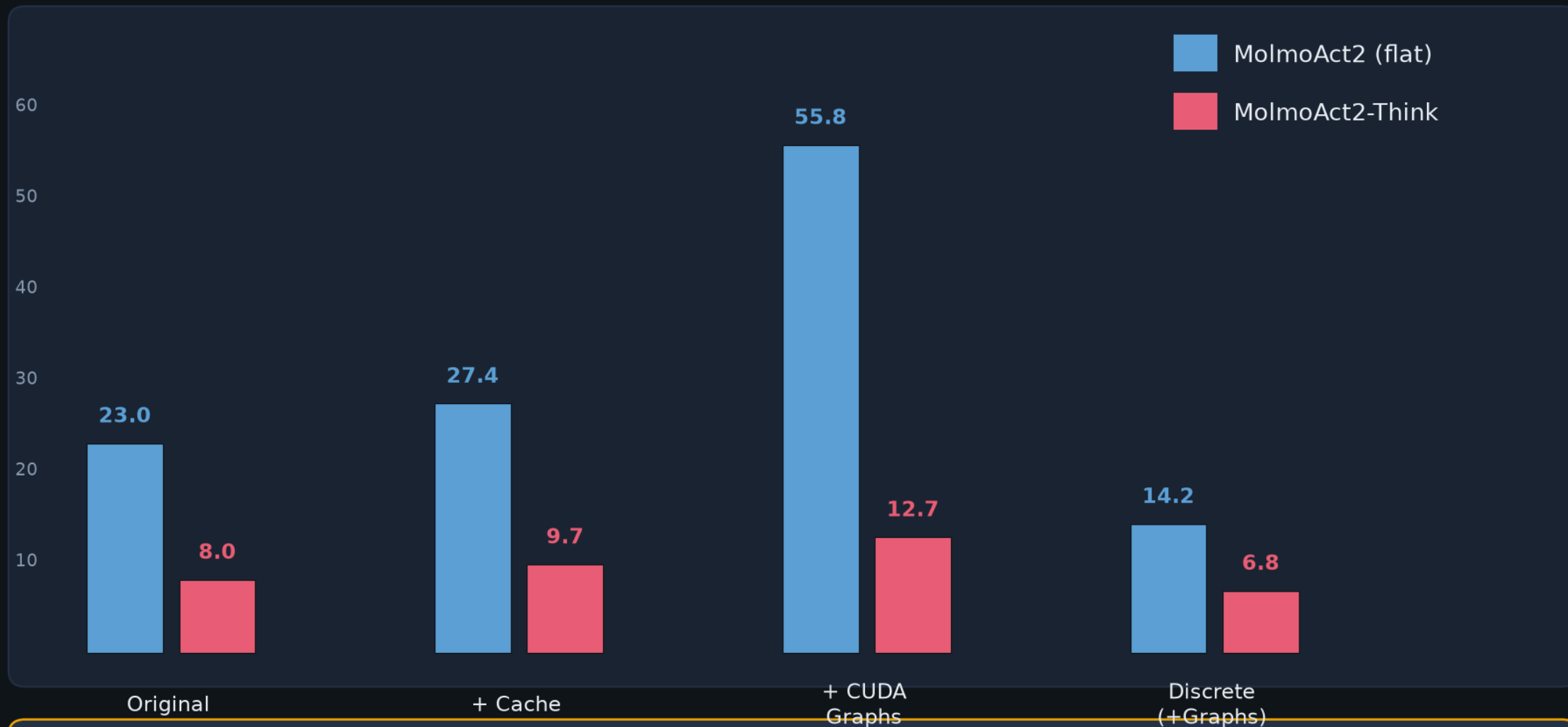
We use annotated language instructions, absolute joint-pose actions, and a 30-step action chunk for the 30 Hz

Parallel lineages — Molmo ARM vs PI stack



Inference control rate — Think on/off + optimizations

$$H=10 \cdot 1 \times H_{100} \cdot \text{LIBERO} \cdot \text{Hz} = H / \text{end-to-end chunk latency}$$



RTC absent — latency via FM cache + CUDA Graphs + Think reuse · don't quote SR without Hz

Visionary · close

2026: keep FM · choose S2 deliberately · hybrid stage+depth · RTC wrap TENTATIVE.

Don't cargo-cult open-loop on household tasks — prefer receding horizon

Q&A · Oct 28 · MolmoAct2 / 2605.02881

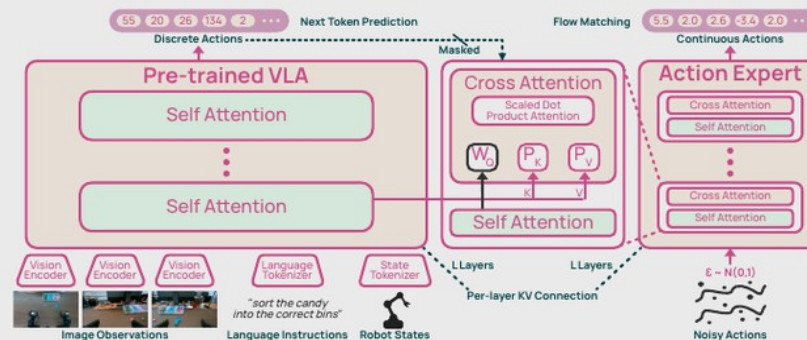


Figure 4 Overview of MolmoAct2. Image observations, language instructions, and robot states are tokenized and processed by a pre-trained VLA backbone with self-attention layers. Post-training attaches a DiT-style action expert with the same number of layers as the backbone. At each layer, the backbone key and value tensors are projected and reused as the key and value inputs to the action expert's cross-attention, enabling per-layer KV conditioning from visual-language context to continuous control. The expert is trained with flow matching to denoise a noisy action trajectory into a continuous robot trajectory. During training, the backbone is also supervised with next-token prediction over discrete action tokens, while the target action-token span is masked from the expert so continuous action prediction cannot condition on the ground-truth discrete actions.

control strings using the same special-token wrappers used during training, e.g., `<setup_start>bimanual yam robotic arms in molmoact2<setup_end>`, and `<control_start>absolute joint pose<control_end>` or `<control_start>delta end-effector pose<control_end>`; the corresponding chat template and output trigger formatting are detailed in Appendix B.

Action and state representation. This stage is deliberately limited to discrete autoregressive supervision: the model predicts action tokens, but does not yet train the continuous action expert used in post-training. Each robot target is represented as a one-second action chunk, so the number of raw actions in the chunk follows the control frequency of the source dataset. Before tokenization, continuous action and state dimensions are normalized with 1-99 percentile statistics; gripper commands are treated separately from this percentile scaling when they are represented as binary or narrow-range open/close signals. Action vectors are padded to a 32-dimensional space and then encoded with the 2048-token vocabulary of MOLMOACT2-FAST TOKENIZER. Proprioceptive state is represented separately: after normalization, each state value is uniformly discretized into one of 256 state tokens and appended to the prompt before the action target.

Training. We train for 200K steps with a maximum sequence length of 4200 tokens. Examples vary substantially in length: a text-only example may use only a few hundred tokens, while multi-camera robot examples must carry images, states, and action targets. We therefore use on-the-fly packing to combine multiple short examples into one 4200-token sequence. Packed examples share the same forward pass, but their text, visual tokens, state tokens, and action targets remain separated by the training attention mask, so the model does not condition one example on another. Appendix B gives the implementation details for this packing procedure and the shared training stack. We train the vision encoder, connector, language model, and newly added tokens' embeddings with a global batch size of 128 across 64 H100 GPUs for around 5,760 GPU hours. The vision encoder and connector use a learning rate of 5×10^{-6} , and the language model uses 1×10^{-5} . This produces a discrete VLA checkpoint that can later be adapted to continuous control.